Introducción a la Programación
Algoritmos y Estructuras de Datos I

Segundo cuatrimestre de 2026

Departamento de Computación - FCEyN - UBA

Introducción a Complejidad Algorítmica

¿Cuánto tiempo demora mi programa?

- Imaginen que tienen el problema de sumar desde 1 hasta n en dos versiones.

```
def sumatoria_1(n : int) → int:
    total : int = 0
    for i in range(1, n):
        total = total + i
    return total

def sumatoria_2(n : int) → int:
    total : int = (n * (n-1)) // 2          # // es la división entera.
    return total
```

- ¿Cuánto demoran? ¿Qué versión es más lenta? ¿Por qué? ¿En cualquier compu? ¿De qué depende? ¿Cómo piensan que cambiaría el tiempo en sumatoria_1 a medida que n aumenta? ¿y en sumatoria_2? ¿Si tengo que pedir una compu prestada para correr mi programa, por cuánto tiempo lo pido? ¿Podremos saberlo de antemano (sin correr nada)?

Objetivos de hoy

- Medir el tiempo de ejecución de programas en Python.
- Entender el conteo de operaciones elementales a nivel práctico e implementativo.
- Continuará (con esteroides) en Algoritmos Y Estructuras de Datos (Algo 2 para LCD).

Importancia para medir tiempos

Medir el tiempo de ejecución de un programa es importante por varias razones:

- **Eficiencia:** Permite evaluar qué tan rápido se ejecuta un programa. En aplicaciones donde el tiempo es crítico, como en sistemas en tiempo real, videojuegos, transacciones financieras o análisis de grandes volúmenes de datos, el rendimiento puede ser un factor decisivo.
- **Optimización:** Identificar las partes del código que consumen más tiempo puede ayudar a optimizar el programa. Al saber dónde se están produciendo los cuellos de botella, los desarrolladores pueden concentrarse en mejorar esas áreas específicas.
- **Comparación de Algoritmos:** Para elegir el mejor algoritmo entre varias alternativas, es útil medir el tiempo de ejecución. Un algoritmo puede ser más eficiente que otro en términos de velocidad, lo que es crucial para aplicaciones donde el rendimiento es importante.
- **Eficiencia de Recursos:** Un programa más rápido generalmente consume menos recursos del sistema, como CPU y memoria, lo que puede ser importante en entornos con recursos limitados.

Medición del tiempo de ejecución

```
import time                                  # Biblioteca de python

def sumatoria_con_tiempos(n : int) → int:
  t0 : float = time.time()                   # segundos desde 1970
  suma = sumatoria(n)                        # Función que nos interesa medir
  tf : float = time.time()

  tiempo_segundos : float = tf - t0          # Tiempo total (en segundos)
  return tiempo_segundos                     # Devolvemos el tiempo (porque sí)


In [49]: sumatoria_con_tiempos(10_000_000)
Out[49]: 1.3811
```

[0]los guiones bajos en python no afectan al número. Solo lo hacen más fácil de leer

Medición del tiempo de ejecución

¿Qué ocurre si ejecutamos esta instrucción múltiples veces?

```
In [50]:  sumatoria_con_tiempos(10_000_000)
Out[50]: 1.2316

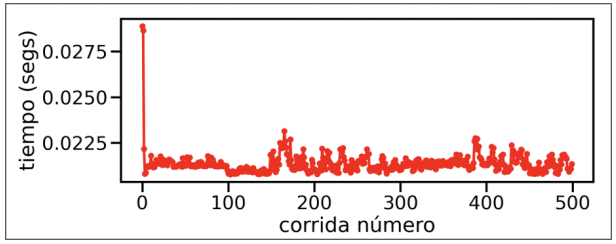
In [51]:  sumatoria_con_tiempos(10_000_000)
Out[51]: 1.2642

In [52]:  sumatoria_con_tiempos(10_000_000)
Out[52]: 1.4275
```


Medición del tiempo de ejecución

Gráfico que muestra 500 corridas de sumatoria_con_tiempos(100_000_000)

```
tiempos_de_ejecución = []
for i in range(500):
    tiempo_i = sumatoria_con_tiempos(1_000_000)
    tiempos_de_ejecución.append(tiempo_i)
graficar(tiempos_de_ejecución)
```



Las primeras corridas parecen llevar más tiempo, luego se estabiliza (más en la materia sistemas operativos)

Cantidad de operaciones elementales

- Una forma de medir la complejidad de una función, es a partir de contar operaciones elementales (sumas, restas, multiplicaciones, divisiones, asignaciones, condicionales, returns)
- Método agnóstico a la computadora. Pero lamentablemente modifica su código:

```
def sumatoria(n : int) → int:
  total : int = 0              # Asignación de total a un valor

  for i in range(1, n):        # Asignación de i a un valor

    total = total + i          # Asignación de total a un valor y la suma de total con

  return total

In [53]: sumatoria(1_000)
Out[53]: 499500
```

Cantidad de operaciones elementales

- Una forma de medir la complejidad de una función, es a partir de contar operaciones elementales (sumas, restas, multiplicaciones, divisiones, asignaciones, condicionales, returns)
- Método agnóstico a la computadora. Pero lamentablemente modifica su código:

```
def sumatoria_con_cuentas(n : int) → int:
  cant_operaciones : int = 0  # (op. agregada, no cuenta)
  total : int = 0             # Asignación de total a un valor
  cant_operaciones += 1       # (op. agregada, no cuenta)

  for i in range(1, n):       # Asignación de i a un valor
    cant_operaciones += 1     # (op. agregada, no cuenta)
    total = total + i         # Asignación de total a un valor y la suma de total con i
    cant_operaciones += 2     # (op. agregada, no cuenta)
  return total, cant_operaciones

In [54]: sumatoria_con_cuentas(1_000)
Out[54]: (499500, 2998)
```


Cantidad de operaciones elementales

- En general mirando el código, podemos hacer un análisis que nos permita conocer la cantidad total de operaciones sin necesidad de ejecutar el código.

```
def sumatoria(n : int) → int:
  total : int = 0

  for i in range(1, n):
    total = total + i
  return total
```

- La cantidad total de operaciones de *sumatoria*(1_000) será: $T_{sum}(1000) = 1 + 999 * 3 = 2,998$
- ¿Cuántas operaciones haremos si ahora lo corremos con 100_000_000? $T_{sum}(100_000_000) = 1 + (100,000,000 - 1) * 3 = 299,999,998$

Cantidad de operaciones elementales

- En general mirando el código, podemos hacer un análisis que nos permita conocer la cantidad total de operaciones sin necesidad de ejecutar el código.

```
def sumatoria(n : int) → int:
  total : int = 0

  for i in range(1, n):
    total = total + i
  return total
```

- Podemos entonces deducir una fórmula general, que depende de los parámetros de la función:
- La cantidad total de operaciones de *sumatoria*(n) será: $T_{sum}(n) = 1 + (n - 1) * 3$

Cantidad de operaciones elementales

- En general mirando el código, podemos hacer un análisis que nos permita conocer la cantidad total de operaciones sin necesidad de ejecutar el código.

```
def f(n : int) → int:
  i = 0
  j = 1000
  while( i < n ):
    i = i + 1
    j = j * 2
  return j
```

- Ejercicio, contar las operaciones elementales de esta función $T_f(n) = ??$
- La cantidad total de operaciones de $f(n)$ será: $T_f(n) = 2 + n * (2 + 2)$